

SECTION 8

EXCEPTION PROCESSING

Exception processing is defined as the activities performed by the processor in preparing to execute a handler routine for any condition that causes an exception. In particular, exception processing does not include execution of the handler routine itself. An introduction to exception processing, as one of the processing states of the MC68030 processor, was given in **SECTION 4 PROCESSING LEVELS**. This section describes exception processing in detail, describing the processing for each type of exception. It describes the return from an exception and bus fault recovery. This section also describes the formats of the exception stack frames. For details of MMU-related exceptions, refer to **SECTION 9 MEMORY MANAGEMENT UNIT**. For more detail on protocol violation and coprocessor-related exceptions, refer to **SECTION 10 COPROCESSOR INTERFACE DESCRIPTION**. Also, for more detail on exceptions defined for floating-point coprocessors, refer to the user's manual for the MC68881/MC68882.

8.1 EXCEPTION PROCESSING SEQUENCE

Exception processing occurs in four functional steps. However, all individual bus cycles associated with exception processing (vector acquisition, stacking, etc.) are not guaranteed to occur in the order in which they are described in this section. Nonetheless, all addresses and offsets from the stack pointer are guaranteed to be as described.

The first step of exception processing involves the status register. The processor makes an internal copy of the status register. Then the processor sets the S bit, changing to the supervisor privilege level. Next, the processor inhibits tracing of the exception handler by clearing the T1 and T0 bits. For the reset and interrupt exceptions, the processor also updates the interrupt priority mask.

In the second step, the processor determines the vector number of the exception. For interrupts, the processor performs an interrupt acknowledge cycle (a read from the CPU address space type \$F; see Figures 7-45 and 7-46) to obtain the vector number. For coprocessor-detected exceptions, the vector number is included in the coprocessor exception primitive response.

(Refer to **SECTION 10 COPROCESSOR INTERFACE DESCRIPTION** for a complete discussion of coprocessor exceptions.) For all other exceptions, internal logic provides the vector number. This vector number is used in the last step to calculate the address of the exception vector. Throughout this section, vector numbers are given in decimal notation.

For all exceptions other than reset, the third step is to save the current processor context. The processor creates an exception stack frame on the active supervisor stack and fills it with context information appropriate for the type of exception. Other information may also be stacked, depending on which exception is being processed and the state of the processor prior to the exception. If the exception is an interrupt and the M bit of the status register is set, the processor clears the M bit in the status register and builds a second stack frame on the interrupt stack.

The last step initiates execution of the exception handler. The processor multiplies the vector number by four to determine the exception vector offset. It adds the offset to the value stored in the vector base register to obtain the memory address of the exception vector. Next, the processor loads the program counter (and the interrupt stack pointer (ISP) for the reset exception) from the exception vector table in memory. After prefetching the first three words to fill the instruction pipe, the processor resumes normal processing at the address in the program counter. Table 8-1 contains a description of all the exception vector offsets defined for the MC68030.

Table 8-1. Exception Vector Assignments (Sheet 1 of 2)

Vector Number(s)	Vector Offset		Assignment	STATUS Asserted
	Hex	Space		
0	000	SP	Reset Initial Interrupt Stack Pointer	—
1	004	SP	Reset Initial Program Counter	—
2	008	SD	Bus Error	Yes
3	00C	SD	Address Error	Yes
4	010	SD	Illegal Instruction	No
5	014	SD	Zero Divide	No
6	018	SD	CHK, CHK2 Instruction	No
7	01C	SD	cpTRAPcc, TRAPcc, TRAPV Instructions	No
8	020	SD	Privilege Violation	No
9	024	SD	Trace	Yes
10	028	SD	Line 1010 Emulator	No
11	02C	SD	Line 1111 Emulator	Yes
12	030	SD	(Unassigned, Reserved)	—
13	034	SD	Coprocessor Protocol Violation	No
14	038	SD	Format Error	No
15	03C	SD	Uninitialized Interrupt	Yes

Table 8-1. Exception Vector Assignments (Sheet 2 of 2)

Vector Number(s)	Vector Offset		Assignment	STATUS Asserted
	Hex	Space		
16 Through 23	040 05C	SD SD	Unassigned, Reserved	—
24 25 26 27	060 064 068 06C	SD SD SD SD	Spurious Interrupt Level 1 Interrupt Autovector Level 2 Interrupt Autovector Level 3 Interrupt Autovector	Yes Yes Yes Yes
28 29 30 31	070 074 078 07C	SD SD SD SD	Level 4 Interrupt Autovector Level 5 Interrupt Autovector Level 6 Interrupt Autovector Level 7 Interrupt Autovector	Yes Yes Yes Yes
32 Through 47	080 0BC	SD SD	TRAP #0–15 Instruction Vectors	No
48 49 50 51	0C0 0C4 0C8 0CC	SD SD SD SD	FPCP Branch or Set on Unordered Condition FPCP Inexact Result FPCP Divide by Zero FPCP Underflow	No No No No
52 53 54 55	0D0 0D4 0D8 0DC	SD SD SD SD	FPCP Operand Error FPCP Overflow FPCP Signaling NAN Unassigned, Reserved	No No No No
56 57 58	0E0 0E4 0E8	SD SD SD	MMU Configuration Error Defined for MC68851 not used by MC68030 Defined for MC68851 not used by MC68030	No — —
59 Through 63	0EC 0FC	SD SD	Unassigned, Reserved	—
64 Through 255	100 3FC	SD SD	User Defined Vectors (192)	Yes

SP = Supervisor Program Space
SD = Supervisor Data Space

As shown in Table 8-1, the first 64 vectors are defined by Motorola and 192 vectors are reserved for interrupt vectors defined by the user. However, external devices may use vectors reserved for internal purposes at the discretion of the system designer.

The MC68030 provides the STATUS signal to identify instruction boundaries and some exceptions. As shown in Table 8-2, STATUS indicates an instruction boundary and exceptions to be processed, depending on the state of the internal microsequencer. In addition, STATUS indicates when an MMU address translation cache miss has occurred and the processor is about to begin a table search access for the logical address that caused the miss. Instruction-related exceptions do not cause the assertion of STATUS as shown in Table 8-1. For STATUS signal timing information, refer to **SECTION 12 APPLICATIONS INFORMATION**.

Table 8-2. Microsequencer STATUS Indications

Asserted for	Indicates
1 Clock	Sequencer at instruction boundary will begin execution of next instruction.
2 Clocks	Sequencer at instruction boundary but will not begin the next instruction immediately due to: • pending trace exception - OR • pending interrupt exception
3 Clocks	MMU address translation cache miss — processor to begin table search OR Exception processing to begin for: • reset OR • bus error OR • address error OR • spurious interrupt OR • autovector interrupt OR • F-line instruction (no coprocessor responded)
Continuously	Processor halted due to double bus fault.

8.1.1 Reset Exception

Assertion by external hardware of the $\overline{\text{RESET}}$ signal causes a reset exception. For details on the requirements for the assertion of $\overline{\text{RESET}}$, refer to **7.8 RESET OPERATION**.

The reset exception has the highest priority of any exception; it provides for system initialization and recovery from catastrophic failure. When reset is recognized, it aborts any processing in progress, and that processing cannot be recovered. Figure 8-1 is a flowchart of the reset exception, which performs the following operations:

1. Clears both trace bits in the status register to disable tracing.
2. Places the processor in the interrupt mode of the supervisor privilege level by setting the supervisor bit and clearing the master bit in the status register.
3. Sets the processor interrupt priority mask to the highest priority level (level 7).
4. Initializes the vector base register to zero (\$00000000).
5. Clears the enable, freeze, and burst enable bits for both on-chip caches and the write-allocate bit for the data cache in the cache control register.
6. Invalidates all entries in the instruction and data caches.
7. Clears the enable bit in the translation control register and the enable bits in both transparent translation registers of the MMU.
8. Generates a vector number to reference the reset exception vector (two long words) at offset zero in the supervisor program address space.
9. Loads the first long word of the reset exception vector into the interrupt stack pointer.
10. Loads the second long word of the reset exception vector into the program counter.

After the initial instruction prefetches, program execution begins at the address in the program counter. The reset exception does not flush the address translation cache (ATC), nor does it save the value of either the program counter or the status register.

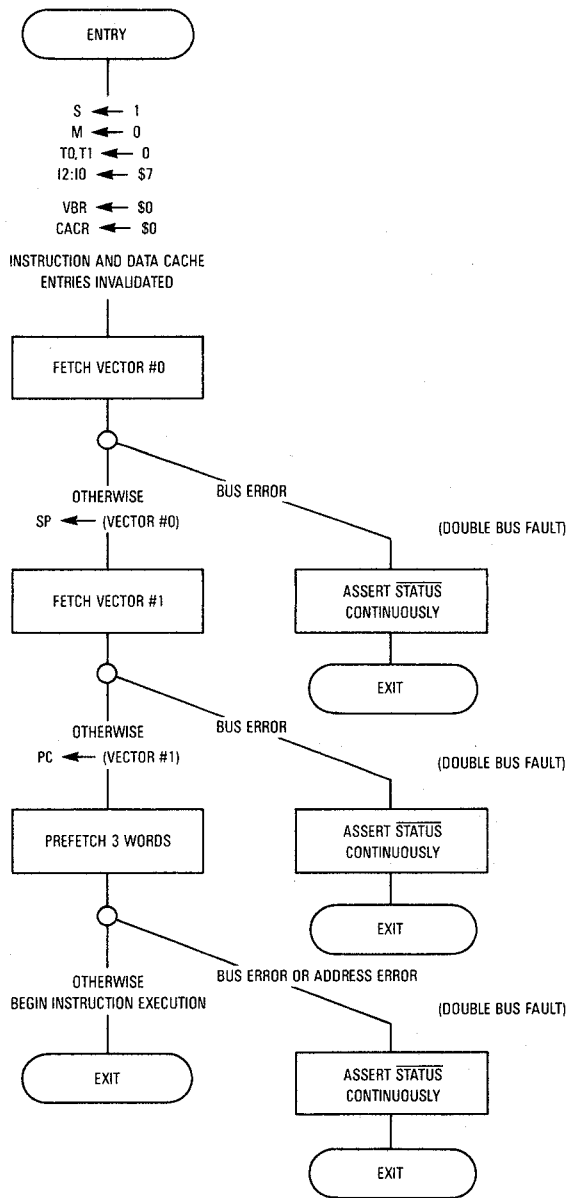


Figure 8-1. Reset Operation Flowchart



As described in **7.5.4 Double Bus Fault**, if bus error or address error occur during the exception processing sequence for a reset, a double bus fault occurs. The processor halts, and the $\overline{\text{STATUS}}$ signal is asserted continuously to indicate the halted condition.

Execution of the reset instruction does not cause a reset exception, nor does it affect any internal registers, but it does cause the MC68030 to assert the $\overline{\text{RESET}}$ signal, resetting all external devices.

8.1.2 Bus Error Exception

A bus error exception occurs when external logic aborts a bus cycle by asserting the $\overline{\text{BERR}}$ input signal. If the aborted bus cycle is a data access, the processor immediately begins exception processing. If the aborted bus cycle is an instruction prefetch, the processor may delay taking the exception until it attempts to use the prefetched information. The assertion of the $\overline{\text{BERR}}$ signal during the second, third, or fourth access of a burst operation does not cause a bus error exception, but the burst is aborted. Refer to **6.1.3.2 BURST MODE FILLING** and **7.5.1 Bus Errors** for details on the effects of bus errors during burst operation.

A bus error exception also occurs when the MMU detects that a successful address translation is not possible. Furthermore, when an ATC miss occurs and an external bus cycle is required, the MMU must abort the bus cycle, search the translation tables in memory for the mapping, and then retry the bus cycle. If a valid translation for the logical address is not available due to a problem encountered during the table search (the attempt to access the appropriate page descriptor in the translation tables for that page), a bus error exception occurs when the aborted bus cycle is retried.

The problem encountered could be a limit violation, an invalid descriptor, or the assertion of the $\overline{\text{BERR}}$ signal during a bus cycle used to access the translation tables. A miss in the ATC causes the processor to automatically initiate a table search but does not cause a bus error exception unless one of the specific conditions mentioned above is encountered.

The processor begins exception processing for a bus error by making an internal copy of the current status register. The processor then enters the supervisor privilege level (by setting the S bit in the status register) and clears the trace bits. The processor generates exception vector number 2 for the bus error vector. It saves the vector offset, program counter, and the internal copy of the status register on the stack. The saved program counter value is the logical address of the instruction that was executing at the time the fault was detected. *This is not necessarily the instruction that initiated the bus cycle, since the processor overlaps execution of instructions.* The processor also saves the contents of some of its internal registers. The information saved on the stack is sufficient to identify the cause of the bus fault and recover from the error.

For efficiency, the MC68030 uses two different bus error stack frame formats. When the bus error exception is taken at an instruction boundary, less information is required to recover from the error, and the processor builds the short bus fault stack frame as shown in Table 8-7. When the exception is taken during the execution of an instruction, the processor must save its entire state for recovery and uses the long bus fault stack frame shown in Table 8-7. The format code in the stack frame distinguishes the two stack frame formats. Stack frame formats are described in detail in **8.4 EXCEPTION STACK FRAME FORMATS**.

If a bus error occurs during the exception processing for a bus error, address error, or reset or while the processor is loading internal state information from the stack during the execution of an RTE instruction, a double bus fault occurs, and the processor enters the halted state as indicated by the continuous assertion of the $\overline{\text{STATUS}}$ signal. In this case, the processor does not attempt to alter the current state of memory. Only an external $\overline{\text{RESET}}$ can restart a processor halted by a double bus fault.

8.1.3 Address Error Exception

An address error exception occurs when the processor attempts to prefetch an instruction from an odd address. This exception is similar to a bus error exception, but is internally initiated. A bus cycle is not executed, and the processor begins exception processing immediately. After exception processing commences, the sequence is the same as that for bus error exceptions described in the preceding paragraphs, except that the vector number is 3 and the vector offset in the stack frame refers to the address error vector. Either a short or long bus fault stack frame may be generated. If an address error occurs during the exception processing for a bus error, address error, or reset, a double bus fault occurs.

8.1.4 Instruction Trap Exception

Certain instructions are used to explicitly cause trap exceptions. The TRAP #n instruction always forces an exception and is useful for implementing system calls in user programs. The TRAPcc, TRAPV, cpTRAPcc, CHK, and CHK2 instructions force exceptions if the user program detects an error, which may be an arithmetic overflow or a subscript value that is out of bounds.

The DIVS and DIVU instructions force exceptions if a division operation is attempted with a divisor of zero.

When a trap exception occurs, the processor copies the status register internally, enters the supervisor privilege level, and clears the trace bits. If tracing is enabled for the instruction that caused the trap, a trace exception is taken after the RTE instruction from the trap handler is executed, and the trace corresponds to the trap instruction; the trap handler routine is not traced. The processor generates a vector number according to the instruction being executed; for the TRAP #n instruction, the vector number is 32 plus n. The stack frame saves the trap vector offset, the program counter, and the internal copy of the status register on the supervisor stack. The saved value of the program counter is the logical address of the instruction following the instruction that caused the trap. For all instruction traps other than TRAP #n, a pointer to the instruction that caused the trap is also saved. Instruction execution resumes at the address in the exception vector after the required instruction prefetches.

8.1.5 Illegal Instruction and Unimplemented Instruction Exceptions

An illegal instruction is an instruction that contains any bit pattern in its first word that does not correspond to the bit pattern of the first word of a valid MC68030 instruction or is a MOVEC instruction with an undefined register specification field in the first extension word. An illegal instruction exception corresponds to vector number 4 and occurs when the processor attempts to execute an illegal instruction.

An illegal instruction exception is also taken if a breakpoint acknowledge bus cycle (see **7.4.2 Breakpoint Acknowledge Cycle**) is terminated with the assertion of the bus error signal. This implies that the external circuitry did not supply an instruction word to replace the BKPT instruction word in the instruction pipe.

Instruction word patterns with bits [15:12] equal to \$A are referred to as unimplemented instructions with A-line opcodes. When the processor attempts to execute an unimplemented instruction with an A-line opcode, an exception is generated with vector number 10, permitting efficient emulation of unimplemented instructions.

Instructions that have word patterns with bits [15:12] equal to \$F, bits [11:9] equal to \$0, and defined word patterns for subsequent words are legal MMU instructions. Instructions that have bits [15:12] of the first words equal to \$F, bits [11:9] equal to \$0, and undefined patterns in subsequent words are treated as unimplemented instructions with F-line opcodes when execution is attempted in supervisor mode. When execution of the same instruction is attempted in user mode, a privilege violation exception is taken. The exception vector number for an unimplemented instruction with an F-line opcode is number 11.

8

The word patterns with bits [15:12] equal to \$F and bits [11:9] not equal to zero are used for coprocessor instructions. When the processor identifies a coprocessor instruction, it runs a bus cycle referencing CPU space type \$2 (refer to **4.2 ADDRESS SPACE TYPES**) and addressing one of seven coprocessors (1–7, according to bits [11:9]). If the addressed coprocessor is not included in the system and the cycle terminates with the assertion of the bus error signal, the instruction takes an unimplemented instruction (F-line opcode) exception. The system can emulate the functions of the coprocessor with an F-line exception handler. Refer to **SECTION 10 COPROCESSOR INTERFACE DESCRIPTION** for more details.

Exception processing for illegal and unimplemented instructions is similar to that for instruction traps. When the processor has identified an illegal or unimplemented instruction, it initiates exception processing instead of attempting to execute the instruction. The processor copies the status register, enters the supervisor privilege level, and clears the trace bits, disabling further tracing. The processor generates the vector number, either 4, 10, or 11, according to the exception type. The illegal or unimplemented instruction vector offset, current program counter, and copy of the status register are saved on the supervisor stack, with the saved value of the program counter being the address of the illegal or unimplemented instruction. Instruction execution resumes at the address contained in the exception vector. It is the responsibility of the handling routine to adjust the stacked program counter if the instruction is emulated in software or is to be skipped on return from the handler.

8.1.6 Privilege Violation Exception

To provide system security, the following instructions are privileged:

- ANDI TO SR
- EOR to SR
- cpRESTORE
- cpSAVE
- MOVE from SR
- MOVE to SR
- MOVE USP
- MOVEC
- MOVES
- ORI to SR
- PFLUSH
- PLOAD
- PMOVE
- PTEST
- RESET
- RTE
- STOP

An attempt to execute one of the privileged instructions while at the user privilege level causes a privilege violation exception. Also, a privilege violation exception occurs if a coprocessor requests a privilege check and the processor is at the user level.

Exception processing for privilege violations is similar to that for illegal instructions. When the processor identifies a privilege violation, it begins exception processing before executing the instruction. The processor copies the status register, enters the supervisor privilege level, and clears the trace bits. The processor generates vector number 8, the privilege violation exception vector, and saves the privilege violation vector offset, the current program counter value, and the internal copy of the status register on the supervisor stack. The saved value of the program counter is the logical address of the first word of the instruction that caused the privilege violation. Instruction execution resumes after the required prefetches from the address in the privilege violation exception vector.

8.1.7 Trace Exception

To aid in program development, the M68000 processors include instruction-by-instruction tracing capability. The MC68030 can be programmed to trace all instructions or only instructions that change program flow. In the trace mode, an instruction generates a trace exception after it completes execution, allowing a debugger program to monitor execution of a program.

8

The T1 and T0 bits in the supervisor portion of the status register control tracing. The state of these bits when an instruction begins execution determines whether the instruction generates a trace exception after the instruction completes. Clearing both T bits disables tracing, and instruction execution proceeds normally. Clearing the T1 bit and setting the T0 bit causes an instruction that forces a change of flow to take a trace exception. Instructions that increment the program counter normally do not take the trace exception. Instructions that are traced in this mode include all branches, jumps, instruction traps, returns, and coprocessor instructions that modify the program counter flow. This mode also includes status register manipulations, because the processor must re-prefetch instruction words to fill the pipe again any time an instruction that can modify the status register is executed. The execution of the BKPT instruction causes a change of flow if the opcode replacing the BKPT is an instruction that causes a change of flow (i.e., a jump, branch, etc.). Setting the T1 bit and clearing the T0 bit causes the execution of all instructions to force trace exceptions. Table 8-3 shows the trace mode selected by each combination of T1 and T0.

Table 8-3. Tracing Control

T1	T0	Tracing Function
0	0	No Tracing
0	1	Trace on Change of Flow (BRA, JMP, etc.)
1	0	Trace on Instruction Execution (Any Instruction)
1	1	Undefined, Reserved

In general terms, a trace exception is an extension to the function of any traced instruction — that is, the execution of a traced instruction is not complete until the trace exception processing is completed. If an instruction does not complete due to a bus error or address error exception, trace exception processing is deferred until after the execution of the suspended instruction is resumed and the instruction execution completes normally. If an interrupt is pending at the completion of an instruction, the trace exception processing occurs before the interrupt exception processing starts. If an instruction forces an exception as part of its normal execution, the forced exception processing occurs before the trace exception is processed. See **8.1.12 Multiple Exceptions** for a more complete discussion of exception priorities.

When the processor is in the trace mode and attempts to execute an illegal or unimplemented instruction, that instruction does not cause a trace exception since it is not executed. This is of particular importance to an instruction emulation routine that performs the instruction function, adjusts the stacked program counter to skip the unimplemented instruction, and returns. Before returning, the trace bits of the status register on the stack should be checked. If tracing is enabled, the trace exception processing should also be emulated for the trace exception handler to account for the emulated instruction.

The exception processing for a trace starts at the end of normal processing for the traced instruction and before the start of the next instruction. The processor makes an internal copy of the status register and enters the supervisor privilege level. It also clears the T0 and T1 bits of the status register, disabling further tracing. The processor supplies vector number 9 for the trace exception and saves the trace exception vector offset, program counter value, and the copy of the status register on the supervisor stack. The saved value of the program counter is the logical address of the next instruction to be executed. Instruction execution resumes after the required prefetches from the address in the trace exception vector.

The STOP instruction does not perform its function when it is traced. A STOP instruction that begins execution with $T1 = 1$ and $T0 = 0$ forces a trace exception after it loads the status register. Upon return from the trace handler routine, execution continues with the instruction following the STOP, and the processor never enters the stopped condition.

8.1.8 Format Error Exception

Just as the processor checks that prefetched instructions are valid, the processor (with the aid of a coprocessor, if needed) also performs some checks of data values for control operations, including the coprocessor state frame format word for a cpRESTORE instruction and the stack frame format for an RTE instruction.

The RTE instruction checks the validity of the stack format code. For long bus cycle fault format frames, the RTE instruction also compares the internal version number of the processor to that contained in the frame at memory location $SP + 54$ ($SP + \$36$). This check ensures that the processor can correctly interpret internal state information from the stack frame.

8

The cpRESTORE instruction passes the format word of the coprocessor state frame to the coprocessor for validation. If the coprocessor does not recognize the format value, it signals the MC68030 to take a format error exception. Refer to **SECTION 10 COPROCESSOR INTERFACE DESCRIPTION** for details of coprocessor-related exceptions.

If any of the checks previously described determine that the format of the stacked data is improper, the instruction generates a format error exception. This exception saves a short format stack frame, generates exception vector number 14, and continues execution at the address in the format exception vector. The stacked program counter value is the logical address of the instruction that detected the format error.

8.1.9 Interrupt Exceptions

When a peripheral device requires the services of the MC68030 or is ready to send information that the processor requires, it may signal the processor to take an interrupt exception. The interrupt exception transfers control to a routine that responds appropriately.

The peripheral device uses the active-low interrupt priority level signals ($\overline{\text{IPL0}}\text{--}\overline{\text{IPL2}}$) to signal an interrupt condition to the processor and to specify the priority of that condition. The three signals encode a value of zero through seven ($\overline{\text{IPL0}}$ is the least significant bit). High levels on all three signals correspond to no interrupt requested (level 0) and low levels on $\overline{\text{IPL0}}\text{--}\overline{\text{IPL2}}$ correspond to interrupt request level 7. Values 1–7 specify one of seven levels of prioritized interrupts; level seven has the highest priority. External circuitry can chain or otherwise merge signals from devices at each level, allowing an unlimited number of devices to interrupt the processor.

The $\overline{\text{IPL0}}\text{--}\overline{\text{IPL2}}$ interrupt signals must maintain the interrupt request level until the MC68030 acknowledges the interrupt to guarantee that the interrupt is recognized. The MC68030 continuously samples the $\overline{\text{IPL0}}\text{--}\overline{\text{IPL2}}$ signals on consecutive falling edges of the processor clock to synchronize and debounce these signals. An interrupt request that is the same for two consecutive falling clock edges is considered a valid input. Although the protocol requires that the request remain until the processor runs an interrupt acknowledge cycle for that interrupt value, an interrupt request that is held for as short a period as two clock cycles could be recognized.

The status register of the MC68030 contains an interrupt priority mask (I2, I1, I0, bits 10–8). The value in the interrupt mask is the highest priority level that the processor ignores. When an interrupt request has a priority higher than the value in the mask, the processor makes the request a pending interrupt. Figure 8-2 is a flowchart of the procedure for making an interrupt pending.

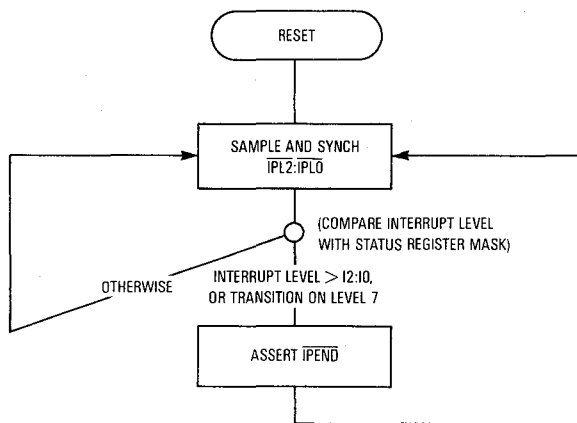


Figure 8-2. Interrupt Pending Procedure

When several devices are connected to the same interrupt level, each device should hold its interrupt priority level constant until its corresponding interrupt acknowledge cycle to ensure that all requests are processed.

Table 8-4 lists the interrupt levels, the states of $\overline{IPL2}$ – $\overline{IPL0}$ that define each level, and the mask value that allows an interrupt at each level.

Table 8-4. Interrupt Levels and Mask Values

Requested Interrupt Level	Control Line Status			Interrupt Mask Level Required for Recognition
	$\overline{IP2}$	$\overline{IP1}$	$\overline{IP0}$	
0*	High	High	High	N/A*
1	High	High	Low	0
2	High	Low	High	0–1
3	High	Low	Low	0–2
4	Low	High	High	0–3
5	Low	High	Low	0–4
6	Low	Low	High	0–5
7	Low	Low	Low	0–7

*Indicates that no interrupt is requested.

Priority level 7, the nonmaskable interrupt (NMI), is a special case. Level 7 interrupts cannot be masked by the interrupt priority mask, and they are transition sensitive. The processor recognizes an interrupt request each time the external interrupt request level changes from some lower level to level 7, regardless of the value in the mask. Figure 8-3 shows two examples of interrupt recognitions, one for level 6 and one for level 7. When the MC68030 processes a level 6 interrupt, the status register mask is automatically updated with a value of 6 before entering the handler routine so that subsequent level 6 interrupts are masked. Provided no instruction that lowers the mask value is executed, the external request can be lowered to level 3 and then raised back to level 6 and a second level 6 interrupt is not processed. However, if the MC68030 is handling a level 7 interrupt (status register mask set to 7) and the external request is lowered to level 3 and then raised back to level 7, a second level 7 interrupt is processed. The second level 7 interrupt is processed because the level 7 interrupt is transition sensitive. A level 7 interrupt is also generated by a level comparison if the request level and mask level are at seven and the priority mask is then set to a lower level (with the MOVE to SR or RTE instruction, for example). As shown in Figure 8-3 for level 6 interrupt request level and mask level, this is the case for all interrupt levels.

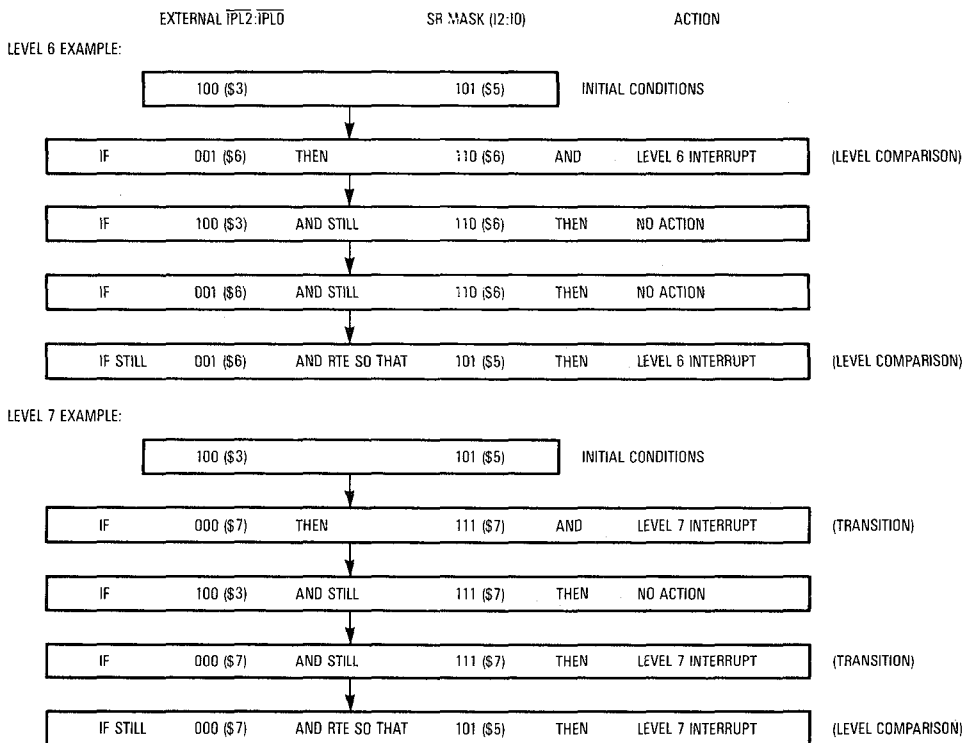


Figure 8-3. Interrupt Recognition Examples

Note that a mask value of 6 and a mask value of 7 both inhibit request levels 1–6 from being recognized. In addition, neither masks a transition to an interrupt request level of 7. The only difference between mask values of 6 and 7 occurs when the interrupt request level is 7 and the mask value is 7. If the mask value is lowered to 6, a second level 7 interrupt is recognized.

The MC68030 asserts the interrupt pending signal ($\overline{IPEND}$) when it makes an interrupt request pending. Figure 8-4 shows the assertion of $\overline{IPEND}$ relative to the assertion of an interrupt level on the IPL lines. $\overline{IPEND}$ signals to external devices that an interrupt exception will be taken at an upcoming instruction boundary (following any higher priority exception).

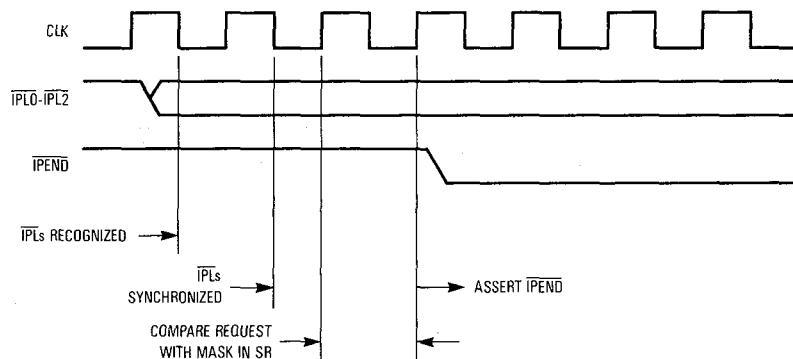


Figure 8-4. Assertion of $\overline{\text{IPEND}}$

The state of the $\overline{\text{IPEND}}$ signal is internally checked by the processor once per instruction, independently of bus operation. In addition, it is checked during the second instruction prefetch associated with exception processing. Figure 8-5 is a flowchart of the interrupt recognition and associated exception processing sequence.

8

To predict the instruction boundary during which a pending interrupt is processed, the timing relationship between the assertion of $\overline{\text{IPEND}}$ for that interrupt and the assertion of $\overline{\text{STATUS}}$ must be examined. Figure 8-6 shows two examples of interrupt recognition. The first assertion of $\overline{\text{STATUS}}$ after $\overline{\text{IPEND}}$ is denoted as STAT0 . The next assertion of $\overline{\text{STATUS}}$ is denoted as STAT1 . If STAT0 begins on the falling edge of the clock immediately following the clock edge that caused $\overline{\text{IPEND}}$ to assert (as shown in example 1), STAT1 is at least two clocks long, and, when there are no other pending exceptions, the interrupt is acknowledged at the boundary defined by STAT1 . If $\overline{\text{IPEND}}$ is asserted with more setup time to STAT0 , the interrupt may be acknowledged at the boundary defined by STAT0 (as shown in example 2). In that case, STAT0 is asserted for two clocks, signaling this condition.

If no higher priority interrupt has been synchronized, the $\overline{\text{IPEND}}$ signal is negated during state 0 (S0) of an interrupt acknowledge cycle (refer to **7.4.1.1 INTERRUPT ACKNOWLEDGE CYCLE — TERMINATED NORMALLY**), and the $\overline{\text{IPLx}}$ signals for the interrupt being acknowledged can be negated at this time.

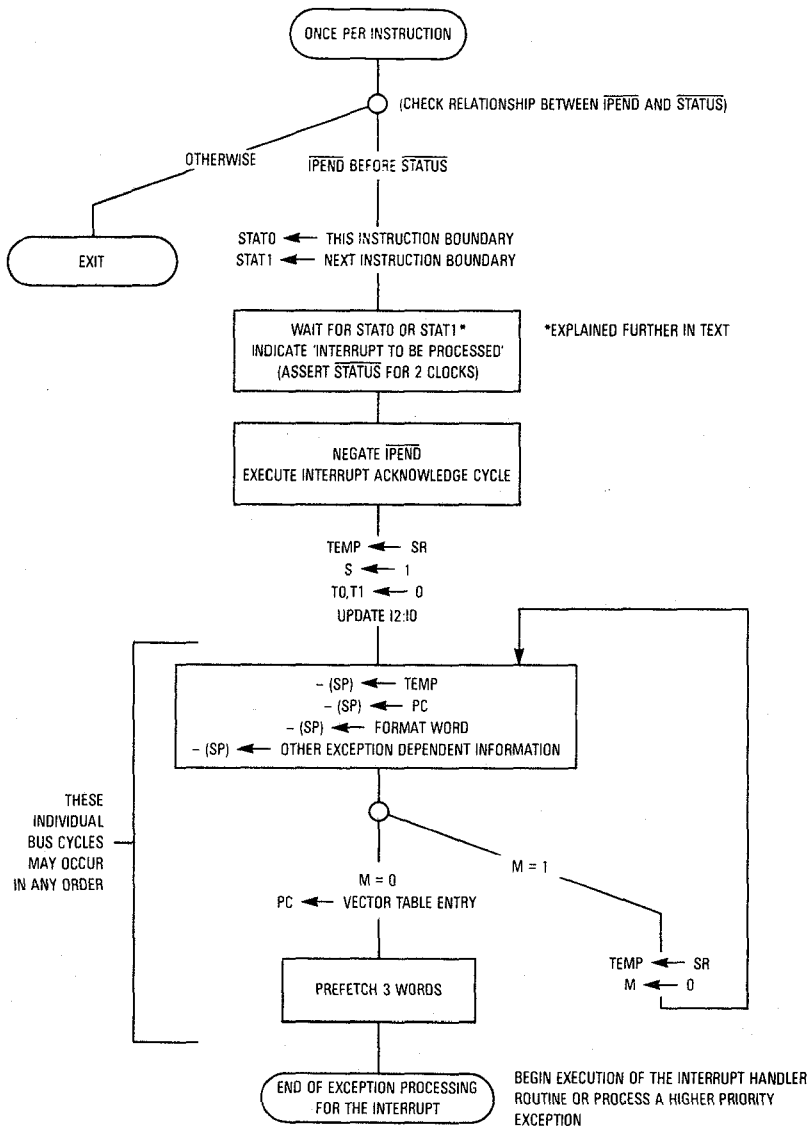
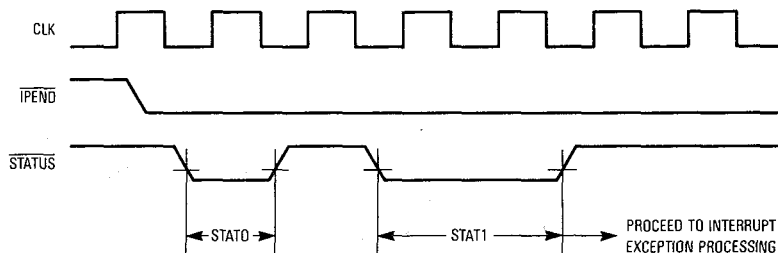
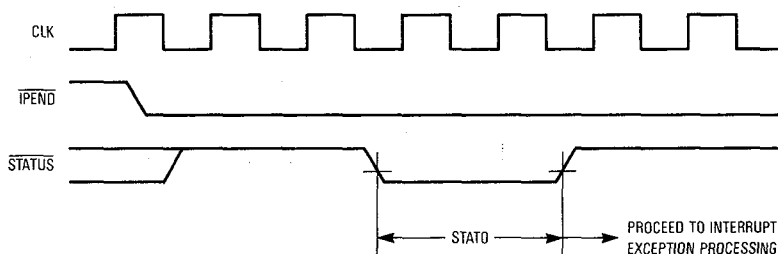


Figure 8-5. Interrupt Exception Processing Flowchart



EXAMPLE 1: INTERRUPT EXCEPTION SIGNED DURING STAT1



EXAMPLE 2: INTERRUPT EXCEPTION SIGNED DURING STAT0

Figure 8-6. Examples of Interrupt Recognition and Instruction Boundaries

When processing an interrupt exception, the processor first makes an internal copy of the status register, sets the privilege level to supervisor, suppresses tracing, and sets the processor interrupt mask level to the level of the interrupt being serviced. The processor attempts to obtain a vector number from the interrupting device using an interrupt acknowledge bus cycle with the interrupt level number output on pins A1–A3 of the address bus. For a device that cannot supply an interrupt vector, the autovector signal ($\overline{AVEC}$) can be asserted, and the MC68030 uses an internally generated autovector, which is one of vector numbers 25–31, that corresponds to the interrupt level number. If external logic indicates a bus error during the interrupt acknowledge cycle, the interrupt is considered spurious, and the processor generates the spurious interrupt vector number, 24. Refer to **7.4.1 Interrupt Acknowledge Bus Cycles** for complete interrupt bus cycle information.

Once the vector number is obtained, the processor saves the exception vector offset, program counter value, and the internal copy of the status register on the active supervisor stack. The saved value of the program counter is the logical address of the instruction that would have been executed had the interrupt not occurred. If the interrupt was acknowledged during the execution of a coprocessor instruction, further internal information is saved on the stack so that the MC68030 can continue executing the coprocessor instruction when the interrupt handler completes execution.

If the M bit of the status register is set, the processor clears the M bit and creates a throwaway exception stack frame on top of the interrupt stack as part of interrupt exception processing. This second frame contains the same program counter value and vector offset as the frame created on top of the master stack, but has a format number of 1 instead of 0 or 9. The copy of the status register saved on the throwaway frame is exactly the same as that placed on the master stack except that the S bit is set in the version placed on the interrupt stack. (It may or may not be set in the copy saved on the master stack.) The resulting status register (after exception processing) has the S bit set and the M bit cleared.

The processor loads the address in the exception vector into the program counter, and normal instruction execution resumes after the required pre-fetches for the interrupt handler routine.

Most M68000 Family peripherals use programmable interrupt vector numbers as part of the interrupt request/acknowledge mechanism of the system. If this vector number is not initialized after reset and the peripheral must acknowledge an interrupt request, the peripheral usually returns the vector number for the uninitialized interrupt vector, 15.

8.1.10 MMU Configuration Exception

When the MC68030 executes a PMOVE instruction that attempts to move invalid data into the TC, CRP, or SRP register of the MMU, the PMOVE instruction causes an MMU configuration exception. The exception is a post-instruction exception; it is processed after the instruction completes. The processor generates exception vector number 56 when an MMU configuration exception occurs. Refer to **SECTION 9 MEMORY MANAGEMENT UNIT** for a description of the valid configurations for the MMU registers.

The processor copies the status register, enters the supervisor privilege level, and clears the trace bits. The processor saves the vector offset, the scanPC value (which points to the next instruction), and the copy of the status register on the supervisor stack. It also saves the logical address of the PMOVE instruction on the stack. Then the processor resumes normal instruction execution after the required prefetches from the address in the exception vector.

8.1.11 Breakpoint Instruction Exception

To use the MC68030 in a hardware emulator, it must provide a means of inserting breakpoints in the emulator code and of performing appropriate operations at each breakpoint. For the MC68000 and MC68008, this can be done by inserting an illegal instruction at the breakpoint and detecting the illegal instruction exception from its vector location. However, since the vector base register on the MC68010, MC68020, and MC68030 allows arbitrary relocation of exception vectors, the exception address cannot reliably identify a breakpoint. The MC68020 and MC68030 processors provide a breakpoint capability with a set of breakpoint instructions, \$4848–\$484F, for eight unique breakpoints. The breakpoint facility also allows external hardware to monitor the execution of a program residing in the on-chip instruction cache without severe performance degradation.

8

When the MC68030 executes a breakpoint instruction, it performs a breakpoint acknowledge cycle (read cycle) from CPU space type \$0 with address lines A2–A4 corresponding to the breakpoint number. Refer to Figure 7-44 for the CPU space type \$0 addresses and to **7.4.2 Breakpoint Acknowledge Cycle** for a description of the breakpoint acknowledge cycle. The external hardware can return either $\overline{\text{BERR}}$, $\overline{\text{DSACKx}}$, or $\overline{\text{STERM}}$ with an instruction word on the data bus. If the bus cycle terminates with $\overline{\text{BERR}}$, the processor performs illegal instruction exception processing. If the bus cycle terminates with $\overline{\text{DSACKx}}$ or $\overline{\text{STERM}}$, the processor uses the data returned to replace the breakpoint instruction in the internal instruction pipe and begins execution of that instruction. The remainder of the pipe remains unaltered. In addition, no stacking or vector fetching is involved with the execution of the instruction. Figure 8-7 is a flowchart of the breakpoint instruction execution.

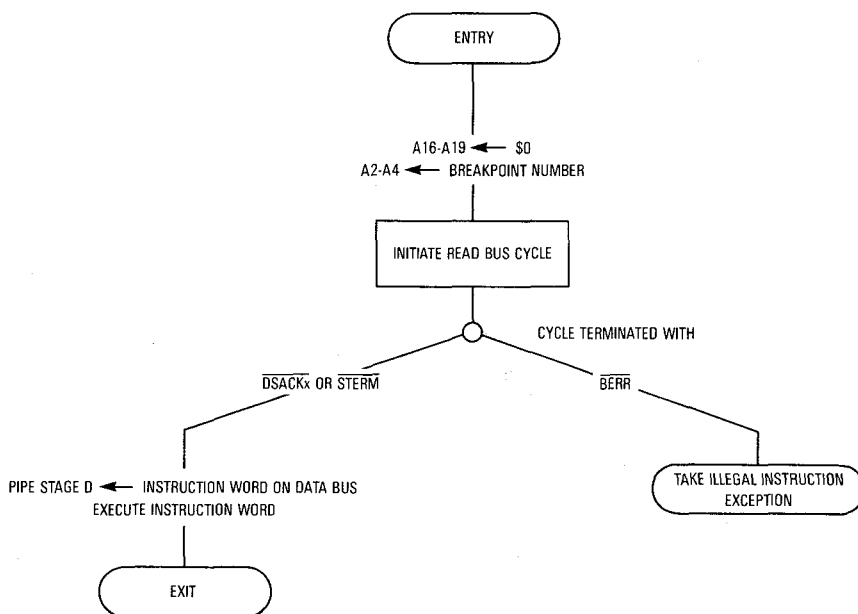


Figure 8-7. Breakpoint Instruction Flowchart

8.1.12 Multiple Exceptions

When several exceptions occur simultaneously, they are processed according to a fixed priority. Table 8-5 lists the exceptions, grouped by characteristics. Each group has a priority from 0–4. Priority 0 has the highest priority.

As soon as the MC68030 has completed exception processing for a condition when another exception is pending, it begins exception processing for the pending exception instead of executing the exception handler for the original exception condition. Also, whenever a bus error or address error occurs, its exception processing takes precedence over lower priority exceptions and occurs immediately. For example, if a bus error occurs during the exception processing for a trace condition, the system processes the bus error and executes its handler before completing the trace exception processing. However, most exceptions cannot occur during exception processing, and very few combinations of the exceptions shown in Table 8-5 can be pending simultaneously.

Table 8-5. Exception Priority Groups

Group/ Priority	Exception and Relative Priority	Characteristics
0	0.0 — Reset	Aborts all processing (instruction or exception) and does not save old context.
1	1.0 — Address Error 1.1 — Bus Error	Suspends processing (instruction or exception) and saves internal context.
2	2.0 — BKPT #n, CHK, CHK2, cp Mid-Instruction, cp Protocol Violation, cp-TRAPcc, Divide by Zero, RTE, TRAP #n, TRAPV, MMU Configuration	Exception processing is part of instruction execution.
3	3.0 — Illegal Instruction, Line A, Unimplemented Line F, Privilege Violation, cp Pre-Instruction	Exception processing begins before instruction is executed.
4	4.0 — cp Post-Instruction 4.1 — Trace 4.2 — Interrupt	Exception processing begins when current instruction or previous exception processing is completed.

0.0 is the highest priority, 4.2 is the lowest.

The priority scheme is very important in determining the order in which exception handlers execute when several exceptions occur at the same time. As a general rule, the lower the priority of an exception, the sooner the handler routine for that exception executes. For example, if simultaneous trap, trace, and interrupt exceptions are pending, the exception processing for the trap occurs first, followed immediately by exception processing for the trace and then for the interrupt. When the processor resumes normal instruction execution, it is in the interrupt handler, which returns to the trace handler, which returns to the trap exception handler. This rule does not apply to the reset exception; its handler is executed first even though it has the highest priority because the reset operation clears all other exceptions.

8.1.13 Return from Exception

After the processor has completed exception processing for all pending exceptions, the processor resumes normal instruction execution at the address in the vector for the last exception processed. Once the exception handler has completed execution, the processor must return to the system context prior to the exception (if possible). The RTE instruction returns from the handler to the previous system context for any exception.

When the processor executes an RTE instruction, it examines the stack frame on top of the active supervisor stack to determine if it is a valid frame and what type of context restoration it requires. This section describes the processing for each of the stack frame types; refer to **8.3 COPROCESSOR CONSIDERATIONS** for a description of the stack frame type formats.

For a normal four-word frame, the processor updates the status register and program counter with the data read from the stack, increments the stack pointer by eight, and resumes normal instruction execution.

For the throwaway four-word stack, the processor reads the status register value from the frame, increments the active stack pointer by eight, updates the status register with the value read from the stack, and then begins RTE processing again, as shown in Figure 8-8. The processor reads a new format word from the stack frame on top of the active stack (which may or may not be the same stack used for the previous operation) and performs the proper operations corresponding to that format. In most cases, the throwaway frame is on the interrupt stack and when the status register value is read from the stack, the S and M bits are set. In that case, there is a normal four-word frame or a ten-word coprocessor mid-instruction frame on the master stack. However, the second frame may be any format (even another throwaway frame) and may reside on any of the three system stacks.

For the six-word stack frame, the processor restores the status register and program counter values from the stack, increments the active supervisor stack pointer by 12, and resumes normal instruction execution.

For the coprocessor mid-instruction stack frame, the processor reads the status register, program counter, instruction address, internal register values, and the evaluated effective address from the stack, restores these values to the corresponding internal registers, and increments the stack pointer by 20. The processor then reads from the response register of the coprocessor that initiated the exception to determine the next operation to be performed. Refer to **SECTION 10 COPROCESSOR INTERFACE DESCRIPTION** for details of coprocessor-related exceptions.

For both the short and long bus fault stack frames, the processor first checks the format value on the stack for validity. In addition, for the long stack frame, the processor compares the version number in the stack with its own version number. The version number is located in the most significant nibble (bits 15–12) of the word at location $SP + \$36$ in the long stack frame. This validity check is required in a multiprocessor system to ensure that the data is properly interpreted by the RTE instruction. The RTE instruction also reads from

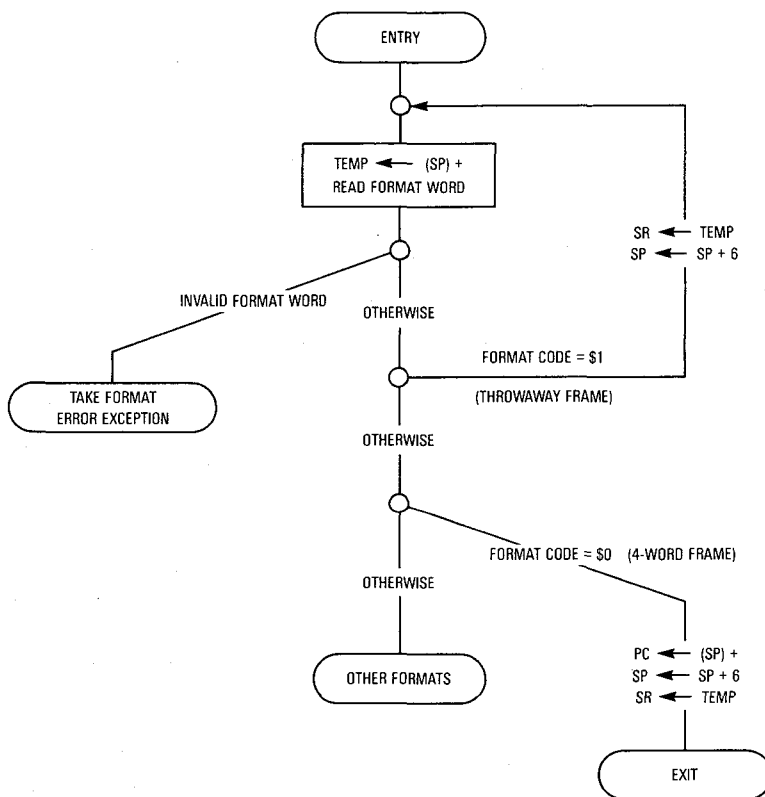


Figure 8-8. RTE Instruction for Throwaway Four-Word Frame

both ends of the stack frame to make sure it is accessible. If the frame is invalid or inaccessible, the processor takes a format error or a bus error exception, respectively. Otherwise, the processor reads the entire frame into the proper internal registers, deallocates the stack, and resumes normal processing. Once the processor begins to load the frame to restore its internal state, the assertion of the $\overline{\text{BERR}}$ signal causes the processor to enter the halted state with the continuous assertion of the STATUS signal. Refer to **8.2 BUS FAULT RECOVERY** for a description of the processing that occurs after the frame is read into the internal registers.

If a format error or bus error exception occurs during the frame validation sequence of the RTE instruction, either due to any of the errors previously described or due to an illegal format code, the processor creates a normal

four-word or a bus fault stack frame below the frame that it was attempting to use. In this way, the faulty stack frame remains intact. The exception handler can examine or repair the faulty frame. In a multiprocessor system, the faulty frame can be left to be used by another processor of a different type (e.g., an MC68010, MC68020, or a future M68000 processor) when appropriate.

8.2 BUS FAULT RECOVERY

An address error exception or a bus error exception indicates a bus fault. The saving of the processor state for a bus error or address error is described in **8.1.2 Bus Error Exception**, and the restoring of the processor state by an RTE instruction is described in **8.1.13 Return from Exception**.

Processor accesses of either data items or the instruction stream can result in bus errors. When a bus error exception occurs while accessing a data item, the exception is taken immediately after the bus cycle terminates. Bus errors reported by the on-chip MMU are also processed immediately. A bus error occurring during an instruction stream access is not processed until the processor attempts to use the information (if ever) that the access should have provided. For instruction faults, when the short format frame applies, the address of the pipe stage B word is the value in the program counter plus four, and the address of the stage C word is the value in the program counter plus two. For the long format, the long word at $SP + \$24$ contains the address of the stage B word; the address of the stage C word is the address of the stage B word minus two. Address error faults occur only for instruction stream accesses, and the exceptions are taken before the bus cycles are attempted.

8.2.1 Special Status Word (SSW)

The internal SSW (see Figure 8-9) is one of several registers saved as part of the bus fault exception stack frame. Both the short bus cycle fault format and the long bus cycle fault format include this word at offset \$A. The bus cycle fault stack frame formats are described in detail at the end of this section.

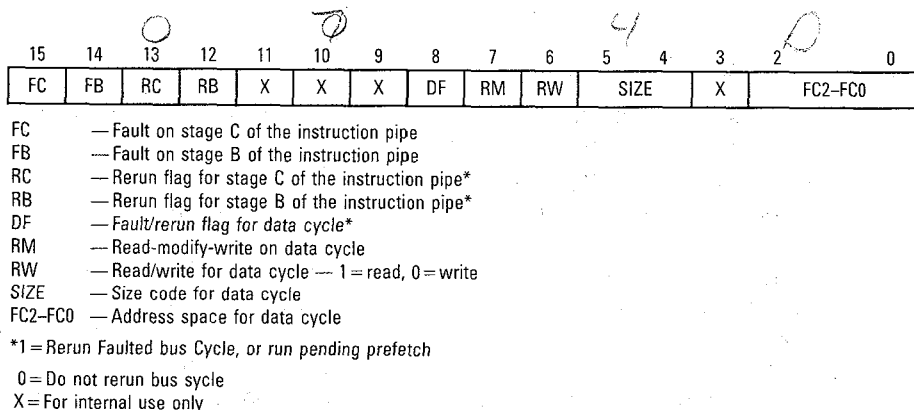


Figure 8-9. Special Status Word (SSW)

The SSW information indicates whether the fault was caused by an access to the instruction stream, data stream, or both. The high-order half of the SSW contains two status bits each for the B and C stages of the instruction pipe. The fault bits (FB and FC) indicate that the processor attempted to use a stage (B or C) and found it to be marked invalid due to a bus error on the prefetch for that stage. The fault bits can be used by a bus error handler to determine the cause(s) of a bus error exception. The rerun flag bits (RB and RC) are set to indicate that a fault occurred during a prefetch for the corresponding stage. A rerun bit is always set when the corresponding fault bit is set. The rerun bits indicate that the word in a stage of the instruction pipe is invalid, and the state of the bits can be used by a handler to repair the values in the pipe after an address error or a bus error, if necessary. If a rerun bit is set when the processor executes an RTE instruction, the processor may execute a bus cycle to prefetch the instruction word for the corresponding stage of the pipe (if it is required). If the rerun and fault bits are set for a stage of the pipe, the RTE instruction automatically reruns the prefetch cycle for that stage. The address space for the bus cycle is the program space for the privilege level indicated in the copy of the status register on the stack. If a rerun bit is cleared, the words on the stack for the corresponding stages

of the pipe are accepted as valid; the processor assumes that there is no prefetch pending for the corresponding stage and that software has repaired or filled the image of the stage, if necessary.

If an address error exception occurs, the fault bits written to the stack frame are not set (they are only set due to a bus error, as previously described), and the rerun bits alone show the cause of the exception. Depending on the state of the pipeline, either RB and RC are both set, or RC alone is set. To correct the pipeline contents and continue execution of the suspended instruction, software must place the correct instruction stream data in the stage C and/or stage B images requested by the rerun bits and clear the rerun bits. The least significant half of the SSW applies to data cycles only. If the DF bit of the SSW is set, a data fault has occurred and caused the exception. If the DF bit is set when the processor reads the stack frame, it reruns the faulted data access; otherwise, it assumes that the data input buffer value on the stack is valid for a read or that the data has been correctly written to memory for a write (or that no data fault occurred). The RM bit of the SSW identifies a read-modify-write operation and the RW bit indicates whether the cycle was a read or write operation. The SIZE field indicates the size of the operand access, and the FC field specifies the address space for the data cycle. Data and instruction stream faults may be pending simultaneously; the fault handler should be able to recognize any combination of the FC, FB, RC, RB, and DF bits.

8.2.2 Using Software To Complete the Bus Cycles

One method of completing a faulted bus cycle is to use a software handler to emulate the cycle. This is the only method for correcting address errors. The handler should emulate the faulted bus cycle in a manner that is transparent to the instruction that caused the fault. For instruction stream faults, the handler may need to run bus cycles for both the B and C stages of the instruction pipe. The RB and RC bits identify the stages that may require a bus cycle; the FB and FC bits indicate that a stage was invalid when an attempt was made to use its contents. Those stages must be repaired. For each faulted stage, the software handler should copy the instruction word from the proper address space as indicated by the S bit of the copy of the status register saved on the stack to the image of the appropriate stage in the stack frame. In addition, the handler must clear the rerun bit associated with the stage that it has corrected. The handler should not change the fault bits FB and FC.

To repair data faults (indicated by $DF = 1$), the software should first examine the RM bit in the SSW to determine if the fault was generated during a read-modify-write operation. If $RM = 0$, the handler should then check the R/W bit of the SSW to determine if the fault was caused by a read or a write cycle. For data write faults, the handler must transfer the properly sized data from the data output buffer (DOB) on the stack frame to the location indicated by the data fault address in the address space defined by the SSW. (Both the DOB and the data fault address are part of the stack frame at $SP + \$18$ and $SP + \$10$, respectively.) Data read faults only generate the long bus fault frame and the handler must transfer properly sized data from the location indicated by the fault address and address space to the image of the data input buffer (DIB) at location $SP + \$2C$ of the long format stack frame. Byte, word, and 3-byte operands are right-justified in the 4-byte data buffers. In addition, the software handler must clear the DF bit of the SSW to indicate that the faulted bus cycle has been corrected.

To emulate a read-modify-write cycle, the exception handler must first read the operation word at the program counter address ($SP + 2$ of the stack frame). This word identifies the CAS, CAS2, or TAS instruction that caused the fault. Then the handler must emulate this entire instruction (which may consist of up to four long word transfers) and update the condition code portion of the status register appropriately, because the RTE instruction expects the entire operation to have been completed if the RM bit is set and the DF bit is cleared. This is true even if the fault occurred on the first read cycle.

To emulate the entire instruction, the handler must save the data and address registers for the instruction (with a MOVEM instruction, for example). Next, the handler reads and modifies (if necessary) the memory location. It clears the DF bit in the SSW of the stack frame and modifies the condition codes in the status register copy and the copies of any data or address registers required for the CAS and CAS2 instructions. Last, the handler restores the registers that it saved at the beginning of the emulation. Except for the data input buffer (DIB), the copy of the status register, and the SSW, the handler should not modify a bus fault stack frame. The only bits in the SSW that may be modified are DF, RB, and RC; all other bits, including those defined for internal use, must remain unchanged.

Address error faults must be repaired in software. Address error faults can be distinguished from bus error faults by the value in the vector offset field of the format word.

8.2.3 Completing the Bus Cycles with RTE

Another method of completing a faulted bus cycle is to allow the processor to rerun the bus cycles during execution of the RTE instruction that terminates the exception handler. This method cannot be used to recover from address errors. The RTE instruction is always executed. Unless the handler routine has corrected the error and cleared the fault (and cleared the rerun and DF bits of the SSW), the RTE instruction can complete the bus cycle(s). If the DF bit is still set at the time of the RTE execution, the faulted data cycle is rerun by the RTE instruction. If the fault bit for a stage of the pipe is set and the corresponding rerun bit was not cleared by the software, the RTE reruns the associated instruction prefetch. The fault occurs again unless the cause of the fault, such as a non-resident page in a virtual memory system, has been corrected. If the rerun bit is set for a stage of the pipe and the fault bit is cleared, the associated prefetch cycle may or may not be run by the RTE instruction (depending on whether the stage is required).

If a fault occurs when the RTE instruction attempts to rerun the bus cycle(s), the processor creates a new stack frame on the supervisor stack after deallocating the previous frame, and address error or bus error exception processing starts in the normal manner.

The read-modify-write operations of the MC68030 can also be completed by the RTE instruction that terminates the handler routine. The rerun operation, executed by the RTE instruction with the DF bit of the SSW set, reruns the entire instruction. If the cause of the error has been corrected, the handler does not need to emulate the instruction but can leave the DF bit set and execute the RTE instruction.

Systems programmers and designers should be aware that the MMU of the MC68030 treats any bus cycle with $\overline{RMC}$ asserted as a write operation for protection checking, regardless of the state of $R/\overline{W}$ signal. Otherwise, the potential for partially destroying system pointers with CAS and CAS2 instructions exists since one portion of the write operation could take place and the remainder be aborted by a bus error.

6.3 COPROCESSOR CONSIDERATIONS

Exception handler programmers should consider carefully whether to save and restore the context of a coprocessor at the beginning and end of handler routines for exceptions that can occur during the execution of a coprocessor instruction (i.e., bus errors, interrupts, and coprocessor-related exceptions). The nature of the coprocessor and the exception handler routine determines whether or not saving the state of one or more coprocessors with the cpSAVE and cpRESTORE instructions is required. If the coprocessor allows multiple coprocessor instructions to be executed concurrently, it may require its state to be saved and restored for all coprocessor-generated exceptions, regardless of whether or not the coprocessor is accessed during the handler routine. The MC68882 floating-point coprocessor is an example of this type of coprocessor. On the other hand, the MC68881 floating-point coprocessor requires FSAVE and FRESTORE instructions within an exception handler routine only if the exception handler itself uses the coprocessor.

8.4 EXCEPTION STACK FRAME FORMATS

The MC68030 provides six different stack frames for exception processing. The set of frames includes the normal four- and six-word stack frames, the four-word throwaway stack frame, the coprocessor mid-instruction stack frame, and the short and long bus fault stack frames.

When the MC68030 writes or reads a stack frame, it uses long-word operand transfers wherever possible. Using a long-word-aligned stack pointer with memory that is on a 32-bit port greatly enhances exception processing performance. The processor does not necessarily read or write the stack frame data in sequential order.

The system software should not depend on a particular exception generating a particular stack frame. For compatibility with future devices, the software should be able to handle any type of stack frame for any type of exception.

Table 8-6 summarizes the stack frames defined for the M68000 Family.

Table 8-6. Exception Stack Frames (Sheet 1 of 2)

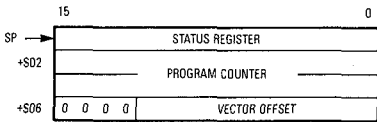
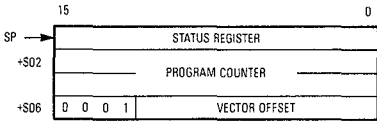
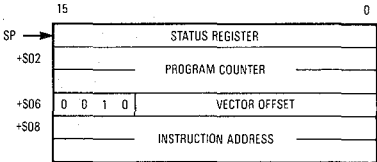
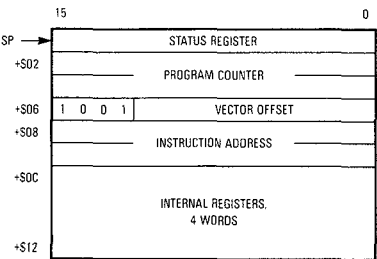
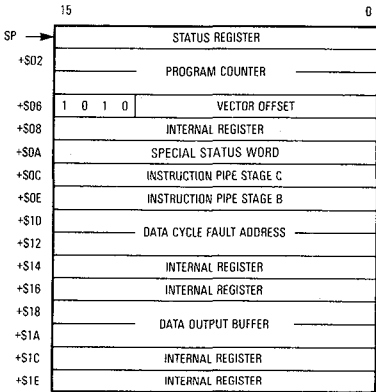
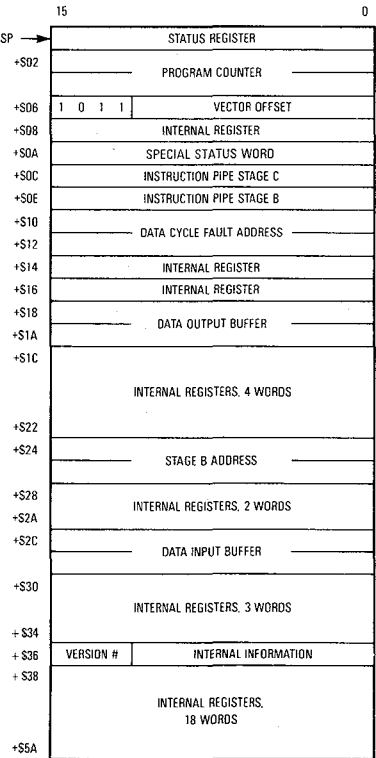
Stack Frames	Exception Types (Stacked PC Points to)
 FOUR WORD STACK FRAME - FORMAT S0	• Interrupt [Next instruction] • Format Error [RTE or cpRESTORE instruction] • TRAP =N [Next instruction] • Illegal Instruction [Illegal instruction] • A-Line Instruction [A-line instruction] • F-Line Instruction [F-line instruction] • Privilege Violation [First word of instruction causing Privilege Violation] • Coprocessor Pre-Instruction [Op-Word of instruction that returned the Take Pre-Instruction primitive]
 THROWAWAY FOUR WORD STACK FRAME - FORMAT S1	• Created on Interrupt Stack [Next instruction — same during interrupt exception as on master stack processing when transition from master state to interrupt state occurs]
 SIX WORD STACK FRAME - FORMAT S2	• CHK [Next instruction for all these exceptions] • CHK2 • cpTRAPcc • TRAPcc • TRAPV • Trace • Zero Divide • MMU Configuration • Coprocessor Post-Instruction INSTRUCTION ADDRESS is the address of the instruction that caused the exception.
 COPROCESSOR MID-INSTRUCTION STACK FRAME (10 WORDS) - FORMAT S9	• Coprocessor Mid-Instruction [Next word to be fetched from instruction stream for all these exceptions] • Main-Detected Protocol Violation • Interrupt Detected During Coprocessor Instruction (supported with 'null come again with interrupts allowed' primitive) INSTRUCTION ADDRESS is the address of the instruction that caused the exception

Table 8-6. Exception Stack Frames (Sheet 2 of 2)

Stack Frames	Exception Types (Stacked PC Points to)
 SHORT BUS CYCLE FAULT STACK FRAME (16 WORDS) - FORMAT SA	Address Error or Bus Error — Execution Unit at Instruction Boundary [Next instruction]
 LONG BUS CYCLE FAULT STACK FRAME (46 WORDS) - FORMAT SB	Address Error or Bus Error — Instruction Execution in Progress [Address of instruction in execution when fault occurred — may not be the instruction that generated the faulted bus cycle]